

Laboratorio 7 - Teoría de la Computación

Ricardo Morales - 22289

José Ángel Morales - 22689

Javier Ovalle - 22103

28 de septiembre de 2026

Problema 1. Conteo de operaciones y clasificación *big-Oh*

Algoritmo A

1. Conteo de operaciones: Analizamos el algoritmo desde sus ciclos anidados hacia afuera:

- **Ciclo interno (k):** Inicia en $k = 1$ y se multiplica por 2 en cada iteración ($k = k \times 2$) hasta n . La cantidad de iteraciones está dada por el menor entero x tal que $2^x > n$. Por lo tanto, el ciclo ejecuta su cuerpo $\approx \lfloor \log_2 n \rfloor + 1$ veces.
- **Ciclo medio (j):** Inicia en $j = 1$ y avanza de 1 en 1 hasta cumplir $j + n/2 \leq n$, lo que equivale a $j \leq n/2$. Esto genera $\approx n/2$ iteraciones.
- **Ciclo externo (i):** Inicia en $i = n/2$ y avanza de 1 en 1 hasta n . Esto genera $\approx n - n/2 + 1 \approx n/2$ iteraciones.

Dado que los límites de los ciclos internos no dependen de las variables contadoras de los ciclos externos, multiplicamos sus iteraciones. El cuerpo (`counter++`, de tiempo c_0) se ejecuta aproximadamente:

$$\left(\frac{n}{2}\right) \times \left(\frac{n}{2}\right) \times (\log_2 n) = \frac{n^2}{4} \log_2 n \text{ veces.}$$

Asignando constantes de tiempo c_i a las declaraciones, inicializaciones y evaluaciones de condiciones, el tiempo total de ejecución tiene la forma polinomial:

$$T(n) = a \cdot n^2 \log_2 n + b \cdot n^2 + c \cdot n + d$$

2. Clasificación asintótica: El término dominante es $n^2 \log_2 n$. Por tanto, $T(n) = O(n^2 \log n)$.

3. Demostración: Por definición formal, $T(n) \in O(n^2 \log n)$ si existen constantes $c > 0$ y $n_0 > 0$ tales que $T(n) \leq c \cdot n^2 \log n$ para todo $n \geq n_0$. Sea $T(n) = an^2 \log_2 n + bn^2 + cn + d$. Para $n \geq 2$, sabemos que $\log_2 n \geq 1$, $n \leq n^2$, y $1 \leq n^2$. Sustituyendo por cotas superiores:

$$T(n) \leq an^2 \log_2 n + bn^2 \log_2 n + cn^2 \log_2 n + dn^2 \log_2 n$$

$$T(n) \leq (a + b + c + d)n^2 \log_2 n$$

Eligiendo $c_{\text{constante}} = a + b + c + d$ y $n_0 = 2$, se cumple que $T(n) \leq c_{\text{constante}} \cdot n^2 \log_2 n$. Queda demostrado que $T(n) \in O(n^2 \log n)$.

Algoritmo B

1. Conteo de operaciones:

- La condición inicial `if (n <= 1) return;` toma un tiempo constante c_1 .
- **Ciclo externo (i):** Itera desde $i = 1$ incrementando de 1 en 1 hasta n . Su número de iteraciones es exactamente n .
- **Ciclo interno (j):** Itera desde $j = 1$ hasta n . Sin embargo, en la primera iteración ($j = 1$) ejecuta un `printf` e inmediatamente encuentra una instrucción `break;`, lo que interrumpe y finaliza el ciclo interno incondicionalmente. Por lo tanto, sin importar el valor de n , el ciclo interno siempre ejecuta exactamente 1 iteración por cada ciclo externo.

El cuerpo del ciclo externo se ejecuta n veces, y realiza operaciones de tiempo constante (la inicialización de j , un `printf` y el `break`). El tiempo total de ejecución es:

$$T(n) = a \cdot n + b$$

2. Clasificación asintótica: El tiempo de ejecución es directamente proporcional a n . Por tanto, $T(n) = O(n)$.

3. Demostración: Para probar que $an + b \in O(n)$, buscamos constantes $c > 0$ y $n_0 > 0$ tales que $an + b \leq c \cdot n$ para todo $n \geq n_0$. Asumiendo $n \geq 1$:

$$\begin{aligned} an + b &\leq an + bn \\ an + b &\leq (a + b)n \end{aligned}$$

Elijiendo la constante $c = a + b$ y $n_0 = 1$, la desigualdad $T(n) \leq c \cdot n$ se cumple. Queda demostrado que $T(n) \in O(n)$.

Algoritmo C

1. Conteo de operaciones:

- **Ciclo interno (j):** Inicia en $j = 1$ y avanza dando "saltos" de 4 en 4 (`j += 4`) hasta llegar a n . El número de veces que ejecuta su cuerpo es aproximadamente $n/4$ (estrictamente $\lfloor \frac{n-1}{4} \rfloor + 1$).
- **Ciclo externo (i):** Inicia en $i = 1$ y avanza de 1 en 1 hasta $n/3$. Su número de iteraciones es $\lfloor n/3 \rfloor$.

Nuevamente, las variables de iteración son independientes. El cuerpo principal (el `printf`, de tiempo constante c_0) se ejecuta:

$$\left(\frac{n}{3}\right) \times \left(\frac{n}{4}\right) = \frac{n^2}{12} \text{ veces.}$$

El tiempo de procesamiento para el control de los ciclos y las sentencias constantes resultan en la siguiente ecuación de tiempo polinomial de grado 2:

$$T(n) = a \cdot \frac{n^2}{12} + b \cdot n + c$$

(donde a, b, c representan la agrupación de las constantes de cada instrucción primitiva).

2. Clasificación asintótica: El término con la mayor tasa de crecimiento es n^2 . Se descartan coeficientes y constantes. Por tanto, $T(n) = O(n^2)$.

3. Demostración: Para probar que $T(n) \in O(n^2)$, debemos demostrar que $\exists c, n_0 > 0 \mid T(n) \leq c \cdot n^2 \forall n \geq n_0$. Partimos de $T(n) = \frac{a}{12}n^2 + bn + c$. Para todo $n \geq 1$, sabemos que $n \leq n^2$ y $1 \leq n^2$. Sustituimos en la ecuación:

$$\begin{aligned} \frac{a}{12}n^2 + bn + c &\leq \frac{a}{12}n^2 + bn^2 + cn^2 \\ T(n) &\leq \left(\frac{a}{12} + b + c\right)n^2 \end{aligned}$$

Tomando la constante $c_{\text{constante}} = \frac{a}{12} + b + c$ y el valor inicial $n_0 = 1$, verificamos que $T(n) \leq c_{\text{constante}} \cdot n^2$. Por consiguiente, queda rigurosamente demostrado que $T(n) \in O(n^2)$.

Problema 3. Demostración de enunciados asintóticos

Supongamos, por definición estándar de análisis de algoritmos, que las funciones $f(n)$, $g(n)$ y $h(n)$ son asintóticamente no negativas.

a. $f(n) = \Theta(g(n)) \Leftrightarrow f(n) = O(g(n)) \wedge f(n) = \Omega(g(n))$

Para demostrar una equivalencia lógica ($\Leftrightarrow$), debemos probar ambas direcciones de la implicación.

1. Demostración hacia adelante ($\Rightarrow$): Si $f(n) = \Theta(g(n))$, por definición existen constantes positivas c_1, c_2 y n_0 tales que para todo $n \geq n_0$:

$$0 \leq c_1g(n) \leq f(n) \leq c_2g(n)$$

De esta desigualdad doble, podemos extraer dos desigualdades individuales válidas para todo $n \geq n_0$:

- $0 \leq f(n) \leq c_2g(n)$, lo cual es exactamente la definición de $f(n) = O(g(n))$.
- $0 \leq c_1g(n) \leq f(n)$, lo cual es exactamente la definición de $f(n) = \Omega(g(n))$.

Por lo tanto, la primera mitad de la implicación es verdadera.

2. Demostración hacia atrás ($\Leftarrow$): Si $f(n) = O(g(n)) \wedge f(n) = \Omega(g(n))$, por las definiciones de cada notación, sabemos que:

- Existen $c_2 > 0$ y $n_1 > 0$ tales que $0 \leq f(n) \leq c_2g(n)$ para todo $n \geq n_1$.
- Existen $c_1 > 0$ y $n_2 > 0$ tales que $0 \leq c_1g(n) \leq f(n)$ para todo $n \geq n_2$.

Si elegimos un $n_0 = \max(n_1, n_2)$, garantizamos que ambas condiciones se cumplan simultáneamente para todo $n \geq n_0$. Al combinarlas, obtenemos:

$$0 \leq c_1g(n) \leq f(n) \leq c_2g(n)$$

Esta es exactamente la definición matemática de Θ . Queda demostrada la equivalencia completa.

b. $\max(f(n), g(n)) = \Theta(f(n) + g(n))$

Para probar que pertenece a Θ , debemos encontrar constantes $c_1, c_2 > 0$ y un n_0 tales que:

$$c_1(f(n) + g(n)) \leq \max(f(n), g(n)) \leq c_2(f(n) + g(n))$$

Cota superior (O): Sabemos que, al ser asintóticamente no negativas, tanto $f(n) \geq 0$ como $g(n) \geq 0$. Por lo tanto, el valor máximo entre las dos funciones nunca puede superar la suma de ambas. Es decir:

$$\max(f(n), g(n)) \leq f(n) + g(n)$$

Esto se cumple para todo n . Aquí nuestra constante es $c_2 = 1$.

Cota inferior (Ω): Por otra parte, la suma de ambas funciones nunca podrá ser mayor al doble del elemento máximo. Es decir:

$$f(n) + g(n) \leq \max(f(n), g(n)) + \max(f(n), g(n))$$

$$f(n) + g(n) \leq 2 \max(f(n), g(n))$$

Dividiendo ambos lados entre 2, obtenemos:

$$\frac{1}{2}(f(n) + g(n)) \leq \max(f(n), g(n))$$

Aquí nuestra constante es $c_1 = \frac{1}{2}$. Uniendo ambas cotas, resulta que para todo $n \geq 1$:

$$\frac{1}{2}(f(n) + g(n)) \leq \max(f(n), g(n)) \leq 1(f(n) + g(n))$$

Como logramos acotarla por abajo y por arriba con múltiplos de $(f(n) + g(n))$, queda demostrado que es $\Theta(f(n) + g(n))$.

c. $f(n) = O(g(n)) \wedge g(n) = O(h(n)) \Rightarrow h(n) = \Omega(f(n))$

Aplicando la definición de *big-Oh* para las dos premisas de la implicación:

1. $f(n) = O(g(n))$ implica que existen $c_1 > 0$ y n_1 tales que $0 \leq f(n) \leq c_1 g(n)$ para todo $n \geq n_1$.
2. $g(n) = O(h(n))$ implica que existen $c_2 > 0$ y n_2 tales que $0 \leq g(n) \leq c_2 h(n)$ para todo $n \geq n_2$.

Sea $n_0 = \max(n_1, n_2)$. Para todo $n \geq n_0$, podemos sustituir $g(n)$ en la primera desigualdad utilizando la segunda:

$$f(n) \leq c_1(c_2 h(n))$$

$$f(n) \leq (c_1 c_2) h(n)$$

Esta última desigualdad demuestra la propiedad de transitividad ($f(n) = O(h(n))$). Ahora, para llegar a lo que nos pide el enunciado, dividimos ambos lados de la inecuación entre el término constante ($c_1 c_2$):

$$\frac{1}{c_1 c_2} f(n) \leq h(n)$$

Dado que c_1 y c_2 son constantes positivas, definimos una nueva constante $c_3 = \frac{1}{c_1 c_2}$. Reordenando la expresión tenemos:

$$0 \leq c_3 f(n) \leq h(n)$$

Esta es exactamente la definición formal de Ω . Por lo tanto, queda demostrado que $h(n) = \Omega(f(n))$.

d. $2^{n+1} = O(2^n)$

Para demostrar que $2^{n+1} = O(2^n)$, debemos encontrar constantes positivas c y n_0 que cumplan la siguiente desigualdad para todo $n \geq n_0$:

$$2^{n+1} \leq c \cdot 2^n$$

Por leyes de los exponentes, podemos reescribir el lado izquierdo de la ecuación separando el exponente de la suma:

$$2^{n+1} = 2^n \cdot 2^1 = 2 \cdot 2^n$$

Sustituyendo esto en la desigualdad original de la definición de *big-Oh*:

$$2 \cdot 2^n \leq c \cdot 2^n$$

Si elegimos la constante $c = 2$ y un $n_0 = 1$, obtenemos:

$$2 \cdot 2^n \leq 2 \cdot 2^n$$

Lo cual es una verdad absoluta para todo $n \geq 1$. Por ende, la constante que suprime el crecimiento asintótico existe y la pertenencia a $O(2^n)$ queda probada.

Problema 2

Laboratorio 7 (parte de código)

Autores:

- Ricardo Morales - 22289
- José Ángel Morales - 22689
- Javier Ovalle - 22103

```
In [1]: import time
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Configuración de gráficos
plt.style.use('seaborn-v0_8-darkgrid')
```

```
In [2]: MAX_TIME_SEC = 60.0 # Límite de 60 segundos por corrida

def function_A(n):
    counter = 0
    start_time = time.perf_counter()

    # Rango de n/2 a n inclusive
    for i in range(n // 2, n + 1):
        j = 1
        while j + n // 2 <= n:
            k = 1
            while k <= n:
                counter += 1
                k *= 2

            # Check de seguridad para N grandes
            if counter % 1000000 == 0:
                if time.perf_counter() - start_time > MAX_TIME_SEC:
                    return counter, True # Timeout alcanzado

            j += 1
    return counter, False

def function_B(n):
    counter = 0
    start_time = time.perf_counter()

    if n <= 1:
        return counter, False

    for i in range(1, n + 1):
        for j in range(1, n + 1):
            counter += 1
```

```

        break # El ciclo interno se interrumpe inmediatamente

    if i % 100000 == 0:
        if time.perf_counter() - start_time > MAX_TIME_SEC:
            return counter, True

    return counter, False

def function_C(n):
    counter = 0
    start_time = time.perf_counter()

    for i in range(1, (n // 3) + 1):
        j = 1
        while j <= n:
            counter += 1
            j += 4

            if counter % 1000000 == 0:
                if time.perf_counter() - start_time > MAX_TIME_SEC:
                    return counter, True

    return counter, False

```

```

In [3]: # Tamaños de entrada definidos en las instrucciones
tamanos_entrada = [1, 10, 100, 1000, 10000, 100000, 1000000]

resultados = {
    'A': {'tiempos': [], 'operaciones': [], 'timeout': []},
    'B': {'tiempos': [], 'operaciones': [], 'timeout': []},
    'C': {'tiempos': [], 'operaciones': [], 'timeout': []}
}

funciones = {'A': function_A, 'B': function_B, 'C': function_C}

print("Iniciando Profiling... (Esto puede tomar un par de minutos)")

for nombre, func in funciones.items():
    print(f"\nEvaluando Algoritmo {nombre}...")
    for n in tamanos_entrada:
        t_inicio = time.perf_counter()
        ops, timeout = func(n)
        t_fin = time.perf_counter()

        tiempo_real = t_fin - t_inicio

        resultados[nombre]['tiempos'].append(tiempo_real)
        resultados[nombre]['operaciones'].append(ops)
        resultados[nombre]['timeout'].append(timeout)

    estado = "[TIMEOUT]" if timeout else "[COMPLETADO]"
    print(f"  N = {n:<10} | Tiempo: {tiempo_real:>8.4f}s | Operaciones: {ops:<10}")

    if timeout:
        print(f"    -> {nombre} abortado en N={n} por exceder los {MAX_TIME_SEC}")
        # Rellenar con NaN las entradas restantes para no dejar arrays desigual
        for n_restante in tamanos_entrada[tamanos_entrada.index(n)+1:]:

```

```

        resultados[nombre]['tiempos'].append(np.nan)
        resultados[nombre]['operaciones'].append(np.nan)
        resultados[nombre]['timeout'].append(True)
    break

print("\nProfiling finalizado.")

```

Iniciando Profiling... (Esto puede tomar un par de minutos)

Evaluando Algoritmo A...

N = 1	Tiempo: 0.0000s	Operaciones: 2	[COMPLETADO]
N = 10	Tiempo: 0.0000s	Operaciones: 120	[COMPLETADO]
N = 100	Tiempo: 0.0012s	Operaciones: 17850	[COMPLETADO]
N = 1000	Tiempo: 0.1881s	Operaciones: 2505000	[COMPLETADO]
N = 10000	Tiempo: 28.5393s	Operaciones: 350070000	[COMPLETADO]
N = 100000	Tiempo: 60.0224s	Operaciones: 873000000	[TIMEOUT]

-> A abortado en N=100000 por exceder los 60.0 segundos.

Evaluando Algoritmo B...

N = 1	Tiempo: 0.0000s	Operaciones: 0	[COMPLETADO]
N = 10	Tiempo: 0.0000s	Operaciones: 10	[COMPLETADO]
N = 100	Tiempo: 0.0000s	Operaciones: 100	[COMPLETADO]
N = 1000	Tiempo: 0.0001s	Operaciones: 1000	[COMPLETADO]
N = 10000	Tiempo: 0.0014s	Operaciones: 10000	[COMPLETADO]
N = 100000	Tiempo: 0.0147s	Operaciones: 100000	[COMPLETADO]
N = 1000000	Tiempo: 0.1469s	Operaciones: 1000000	[COMPLETADO]

Evaluando Algoritmo C...

N = 1	Tiempo: 0.0000s	Operaciones: 0	[COMPLETADO]
N = 10	Tiempo: 0.0000s	Operaciones: 9	[COMPLETADO]
N = 100	Tiempo: 0.0000s	Operaciones: 825	[COMPLETADO]
N = 1000	Tiempo: 0.0052s	Operaciones: 83250	[COMPLETADO]
N = 10000	Tiempo: 0.5222s	Operaciones: 8332500	[COMPLETADO]
N = 100000	Tiempo: 56.0801s	Operaciones: 833325000	[COMPLETADO]
N = 1000000	Tiempo: 60.0427s	Operaciones: 920000000	[TIMEOUT]

-> C abortado en N=1000000 por exceder los 60.0 segundos.

Profiling finalizado.

```

In [4]: # Estructuración de los datos en un DataFrame multicapa de Pandas
        columnas = pd.MultiIndex.from_product(
            [['Algoritmo A', 'Algoritmo B', 'Algoritmo C'], ['Tiempo (s)', 'Operaciones']],
            names=['Algoritmo', 'Metrica']
        )

        df_resultados = pd.DataFrame(index=tamano_entrada, columns=columnas)
        df_resultados.index.name = 'Tamano Entrada (N)'

        for nombre in ['A', 'B', 'C']:
            algo_key = f'Algoritmo {nombre}'
            df_resultados[(algo_key, 'Tiempo (s)')] = resultados[nombre]['tiempos']
            df_resultados[(algo_key, 'Operaciones')] = resultados[nombre]['operaciones']

        # Rellenar los valores vacíos (causados por los Timeouts) con texto descriptivo
        df_tabla_visual = df_resultados.fillna('> 60.00s (Timeout)')

```

```
print("\n=== TABLA DE RESULTADOS DE PROFILING ===")
display(df_tabla_visual)
```

=== TABLA DE RESULTADOS DE PROFILING ===

Algoritmo		Algoritmo A		Algoritmo B		Algoritmo C	
Metrica	Tiempo (s)	Operaciones	Tiempo (s)	Operaciones	Tiempo (s)	Operaciones	
Tamano Entrada (N)							
1	0.000006	2.0	0.000001	0	0.000004	0	
10	0.00001	120.0	0.000006	10	0.000003	9	
100	0.001216	17850.0	0.000012	100	0.000049	825	
1000	0.188099	2505000.0	0.000136	1000	0.005219	83250	
10000	28.539323	350070000.0	0.001375	10000	0.522186	8332500	
100000	60.022395	873000000.0	0.014660	100000	56.080096	833325000	
1000000	> 60.00s (Timeout)	> 60.00s (Timeout)	0.146937	1000000	60.042710	920000000	

```
In [5]: fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle('Comparación de Tiempo Real vs Conteo de Operaciones', fontsize=16, y=
n_sizes_arr = np.array(tamanos_entrada)

for i, nombre in enumerate(['A', 'B', 'C']):
    ax1 = axes[i]

    # Extraer datos ignorando los NaN del timeout
    tiempos = np.array(resultados[nombre]['tiempos'])
    ops = np.array(resultados[nombre]['operaciones'])

    mask = ~np.isnan(tiempos)
    valid_n = n_sizes_arr[mask]
    valid_tiempos = tiempos[mask]
    valid_ops = ops[mask]

    # Eje Y Primario: Tiempo
    color1 = 'tab:red'
    ax1.set_xlabel('Tamaño de Entrada (N) [Escala Log]')
    ax1.set_ylabel('Tiempo Real (segundos)', color=color1)
    ax1.plot(valid_n, valid_tiempos, color=color1, marker='o', linewidth=2, label='')
    ax1.tick_params(axis='y', labelcolor=color1)
    ax1.set_xscale('log')

    if np.max(valid_tiempos) > 0:
        ax1.set_yscale('log')

    # Eje Y Secundario: Operaciones
    ax2 = ax1.twinx()
```

```

color2 = 'tab:blue'
ax2.set_ylabel('Conteo de Operaciones', color=color2)
ax2.plot(valid_n, valid_ops, color=color2, marker='s', linestyle='--', linewidth=2)
ax2.tick_params(axis='y', labelcolor=color2)
if np.max(valid_ops) > 0:
    ax2.set_yscale('log')

ax1.set_title(f'Algoritmo {nombre}', fontsize=14, fontweight='bold')
ax1.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

Comparación de Tiempo Real vs Conteo de Operaciones

